# Übungsblatt 3: Interrupts und Timer

**Mikrocomputertechnik 3 · 36 Orientierungspunkte · ca. 75–90 Minuten**

**Musterlösung**

## Aufgabe 1: Ereignisbehandlung auswählen (4 Punkte)

1. Ein Füllstandssensor hält seine Bereitschaft bis zur Abfrage gespeichert. Erkennung spätestens 1,2 ms nach dem Ereignis genügt. Eine Hauptschleife kann die Statusbeobachtungen mit höchstens 900 $\mu$s Abstand durchführen; weitere Verzögerungen sind ausgeschlossen. Empfehlen Sie Polling oder Interrupt und begründen Sie mit einer Zeitgrenze, ob ein Interrupt für diese Anforderung nötig ist. (2 P)
2. Separater Fall: Jeder Schritt eines Förderbands muss gezählt werden. Die 35 $\mu$s langen Impulse liegen mindestens 80 $\mu$s auseinander. Software kann den Eingang nur alle zwei ms beobachten. Zur Wahl stehen ein gespeichertes Bereitschaftsbit und ein Hardwarezähler, der jeden Impuls erfasst und modulo $2^{16}$ zählt. Wählen Sie eine Lösung und begründen Sie deren Voraussetzung für eine eindeutige Zählung zwischen zwei Beobachtungen. Encoder-Modus-Kenntnisse werden nicht vorausgesetzt. (2 P)

**Musterlösung 1**

1. Polling ist unter den genannten Anforderungen ausreichend: höchstens 900 $\mu$s bis zur Beobachtung liegen unter 1200 $\mu$s. Die Bereitschaft bleibt erhalten. Ein Interrupt ist dafür nicht erforderlich. Eine alternative Empfehlung ist bei nachvollziehbaren zusätzlichen Zielen zulässig, etwa dem Vermeiden laufender Abfragen; diese Ziele und die erforderlichen Zeitgrenzen müssen genannt werden. (2 P)
2. Den Hardwarezähler wählen: Das Bit erhält nur „mindestens ein Schritt“, nicht die Anzahl mehrerer Schritte. Den Zählerstand konsistent beobachten und die Differenz modulo $2^{16}$ bilden. Zwischen zwei Beobachtungen dürfen keine $2^{16}$ Schritte unbemerkt vergehen. Bei mindestens 80 $\mu$s Abstand benötigt ein voller Zählumlauf mindestens $65536 \cdot 80\,\mu\text{s} = 5{,}24288$ s, deutlich länger als zwei ms. Die Hardwareerfassung jedes Impulses ist ausdrücklich vorgegeben. (2 P)

## Aufgabe 2: Ursache, Einsprung und Vorbereitung (7 Punkte)

RV32-Lehrmodell: ein Hart im Machine Mode, keine Weiterleitung an andere Privilegstufen und keine verschachtelten Interrupts. Ein Hart ist ein Hardware-Ausführungskontext. Nach einem Trap steht in `mcause` der Rohwert `0x0000000B`.

1. Bestimmen Sie Interrupt-Bit und Code und benennen Sie die Ursache. (1 P)
2. Vectored-Modus, BASE `0x80400800`: Ein Kollege behauptet, der erste Befehl liege bei `0x8040082C`. Prüfen Sie die Behauptung für den Rohwert und begründen Sie die richtige Adresse. Wohin ginge stattdessen ein externer Interrupt mit Code elf? (2 P)
3. Wer erhält jeweils den nötigen Zustand: Hardware oder Software? Gefragt sind die Fortsetzungsadresse in `mepc` und der Wert von `t0`, den die Behandlung verändert, das Hauptprogramm aber noch benötigt. Begründen Sie auch den Umgang mit dem temporären Register. (2 P)
4. Vor dem ersten Timer-Interrupt ist die globale Freigabe MIE null. Ordnen Sie die Schritte zu einer geeigneten Initialisierung und erläutern Sie das Risiko einer zu frühen globalen Freigabe. (2 P)

   - A: Timerkategorie MTIE freigeben.
   - B: Gültigen Stack und Trap-Eintritt über mtvec bereitstellen.
   - C: Globale MIE-Freigabe setzen.
   - D: Zeit konsistent lesen und ersten Zukunftstermin nach mtimecmp schreiben.

**Musterlösung 2**

1. Interrupt-Bit null, Code elf: eine `ecall`-Exception aus dem Machine Mode. Die gleiche Codezahl wie bei Machine External macht sie nicht zum Interrupt. (1 P)
2. Die Exception geht auch in Vectored nach BASE `0x80400800`. Nur der externe Interrupt mit Code elf geht nach $\texttt{0x80400800} + 4 \cdot 11 = \texttt{0x8040082C}$. (2 P)
3. Die Hardware hält die Fortsetzungsadresse in `mepc` fest. (1 P) Software muss den benötigten Wert von `t0` sichern und wiederherstellen. „Temporär“ erlaubt das Verändern bei normalen Funktionsaufrufen, ersetzt aber keine Sicherung bei asynchroner Unterbrechung. (1 P)
4. Eine geeignete Reihenfolge ist B, D, A, C: erst einen gültigen Behandlungspfad und Zukunftstermin bereitstellen, zuletzt global freigeben. Andere unter globaler Sperre sichere Reihenfolgen sind ebenfalls zulässig; entscheidend ist, dass die Vorbereitungen vor C abgeschlossen sind. (1 P) Eine bereits wartende Anforderung könnte sonst in unvorbereitete Software eintreten, etwa ohne gültigen Stack oder Trap-Eintritt. (1 P)

## Aufgabe 3: Gerätebehandlung vollständig machen (6 Punkte)

Ein PLIC-Kontext, alle Quellen wartend und freigegeben, Schwellenwert null. PLIC bedeutet Platform-Level Interrupt Controller. Höhere Priorität gewinnt, bei Gleichstand die kleinere Geräte-ID.

| ID | Quelle | Priorität |
|---|---|---|
| 9 | serielle Schnittstelle | 4 |
| 12 | Beschleunigungssensor | 8 |
| 5 | Drehgeber | 8 |

1. Nennen Sie die ersten beiden Claim-IDs. Nach jedem Claim wird vollständig behandelt und abgeschlossen; keine neuen Ereignisse. Begründen Sie. (2 P)
2. Was ändert Claim am Controller? Begründen Sie, ob damit auch die Ursache im Gerät beseitigt ist. (1 P)
3. Ein fehlerhafter Handler führt nur Claim und Complete aus. Das Gerät hält seine Level-Anforderung weiter aktiv. Ergänzen Sie den fehlenden Schritt und erklären Sie die mögliche Folge. (2 P)
4. Ein späterer Claim liefert null. Welche Gerätebearbeitung und welches Complete sind dann vorgesehen? Begründen Sie. (1 P)

**Musterlösung 3**

1. ID fünf, danach zwölf. Beide haben Priorität acht, im Gleichstand gewinnt fünf. (2 P)
2. Claim übernimmt eine Quelle und löscht atomar deren PLIC-Pending. Die Geräteursache kann weiter bestehen, weil kein Gerätestatus und keine Nutzdaten durch diesen Controllerzugriff behandelt werden. (1 P)
3. Zwischen Claim und Complete Daten beziehungsweise Status nach Gerätevertrag bearbeiten und die Ursache passend quittieren. Eine weiter aktive Level-Ursache kann nach Complete erneut gemeldet werden. (2 P)
4. Keine Gerätebearbeitung und kein Complete für ID null, denn es wurde keine Quelle übernommen. (1 P)

## Aufgabe 4: Fehler im Softwaretick (8 Punkte)

Hardwarezeit mtime: zwölf MHz, Prozessortakt: 96 MHz, Sollperiode 2,5 ms. Der nächste Solltermin und mtimecmp stehen anfangs auf 240000 Timer-Ticks. Kein Zählerumlauf oder arithmetischer Überlauf.

C-Lehrschema für den ISR-Rumpf: Ein- und Austritt sowie Kontextsicherung sind bereitgestellt. Die angegebenen Schnittstellen lesen die Zeit konsistent und schreiben den Vergleich korrekt. `period` ist die konstante Sollperiode in Ticks. Eine Softwareuhr berechnet ihre Zeit aus `periods_seen` mal 2,5 ms.

```
#include <stdint.h>
uint64_t read_mtime(void);
void write_mtimecmp(uint64_t value);
volatile uint32_t periods_seen;
static uint64_t deadline = 240000u;
void timer_work(uint64_t period) {
    ++periods_seen;
    uint64_t now = read_mtime();
    deadline = now + period;
    write_mtimecmp(deadline);
}
```

1. Berechnen Sie `period`. Geht der Prozessortakt in diese Rechnung ein? (1 P)
2. Nennen Sie zwei voneinander zu unterscheidende Ursachen dafür, dass die Softwareuhr nachgeht: bei wiederholter Eintrittsverspätung und nach langen Phasen ohne Interruptbehandlung. (2 P)
3. Statt des fehlerhaften Codes soll das Sollraster erhalten bleiben. Separater Fall: now 346000 bei bisherigem Termin 240000. Bestimmen Sie die Anzahl fälliger Termine und den ersten zukünftigen Rastertermin in Ticks und Millisekunden. (2 P)
4. Korrigieren Sie den ISR-Rumpf. Die Funktion `next_future` aus Folie 35 darf verwendet werden. Geben Sie außerdem an, wie die Anwendung ihre verstrichene Zeit zuverlässig bestimmt. (2 P)
5. Der Sensor hält nur seinen neuesten Messwert bereit. Können wiederholte aktuelle Lesungen die historischen Werte der versäumten Termine ersetzen? Begründen Sie. (1 P)

**Musterlösung 4**

1. $12 \cdot 10^6 \cdot 0{,}0025 = 30000$ Ticks. Der separat vorgegebene Prozessortakt geht nicht ein. (1 P)

2. `deadline = now + period` übernimmt jede Eintrittsverspätung in die Phase der folgenden Termine; wiederholt ergibt das Drift. (1 P) `periods_seen` zählt Aufrufe statt verstrichener Sollperioden. Mehrere fällige Termine müssen nicht mehrere Aufrufe erzeugen; `volatile` ändert daran nichts. (1 P)

3. $k = \lfloor (346000 - 240000)/30000 \rfloor + 1 = 4$. Fällig sind 20, 22,5, 25 und 27,5 ms. (1 P) Nächster Termin: $240000 + 4 \cdot 30000 = 360000$ Ticks = 30 ms. (1 P)

4. Die Schnittstellen und der vorbereitete Anfangstermin bleiben wie angegeben. Der korrigierte Rumpf lautet: (1 P)

```
void timer_work(uint64_t period) {
    uint64_t now = read_mtime();
    deadline = next_future(deadline, now, period);
    write_mtimecmp(deadline);
}
```

   Verstrichene Zeit wird aus einer konsistent gelesenen Hardwarezeit minus einem gespeicherten Startwert gewonnen und über die Timerfrequenz umgerechnet. `periods_seen` entfällt als Zeitbasis. Bei Weitergabe eines mehrteiligen Snapshots ist zusätzlich eine konsistente Übergabe nötig. (1 P)

5. Nein. Aktuelles Lesen rekonstruiert keine zuvor ungespeicherten Werte. Erforderlich wäre ihre rechtzeitige Erfassung und Speicherung. (1 P)

## Aufgabe 5: Priorität, Frist und Dauerbetrieb (6 Punkte)

Ein Hart, ein PLIC-Kontext, nur Sensor (Priorität sieben) und UART (Priorität drei), beide freigegeben, Schwellenwert null, keine Verschachtelung. UART bedeutet Universal Asynchronous Receiver/Transmitter.

Das Hauptprogramm sperrt Interrupts höchstens $20\,\mu$s am Stück. Eine UART-Behandlung belegt einschließlich Ein- und Austritt höchstens $50\,\mu$s. Nach Ende einer Sperre oder Behandlung wird die höchstpriorisierte wartende Quelle vor weiterer Hauptprogrammarbeit übernommen. Für ein einzelnes neues Sensorereignis braucht der Eintritt höchstens $4\,\mu$s, anschließend die Datensicherung weitere $6\,\mu$s. Daten müssen spätestens $50\,\mu$s nach dem Ereignis gesichert sein. Andere Verzögerungen und eine bereits laufende Sensorbehandlung sind ausgeschlossen.

1. Zeichnen Sie den ungünstigsten Weg bis zur Datensicherung. Begründen Sie insbesondere, ob die zwei möglichen Blockierungen addiert werden müssen. Ist die Frist eingehalten? (2 P)
2. Ein Kollege möchte die Sensorpriorität auf das Maximum erhöhen. Bewerten Sie, ob dies die gefundene Fristverletzung beseitigt. (1 P)
3. Welche maximale UART-Behandlungsdauer wäre rechnerisch noch zulässig? Welche Arbeit sollte die UART-Behandlung dafür ins Hauptprogramm abgeben? (2 P)
4. Separat zum Einzelfall treffen im Dauerbetrieb alle 0,8 ms neue Messwerte ein. Die Auswertung eines Werts belegt den Prozessor bereits eine ms. Kann ein endlicher Puffer diesen Betrieb dauerhaft verlustfrei machen? Begründen Sie anhand der Raten. (1 P)

**Musterlösung 5**

1. Größte Blockierung $\max(20, 50) = 50\,\mu$s, nicht siebzig. (1 P) Während der Hauptprogrammsperre beginnt keine UART-Behandlung. Danach gewinnt der bereits wartende Sensor die Auswahl. Nach einer laufenden UART-Behandlung wird ebenfalls der Sensor vor neuer Hauptprogrammarbeit bedient. Die ungünstigste Restdauer ist daher entweder die Sperre oder die UART-Behandlung. Gesamt: $50 + 4 + 6 = 60\,\mu$s; die Frist von fünfzig Mikrosekunden ist verletzt. (1 P)
2. Nein. Der Sensor hat schon die höhere Priorität, kann die bereits laufende UART-Behandlung im nicht verschachtelten Modell aber nicht unterbrechen. (1 P)
3. $\max(20, T_{\mathrm{UART}}) + 4 + 6 \leq 50$ verlangt $T_{\mathrm{UART}} \leq 40\,\mu$s. Der Grenzfall hat keine Zeitreserve. (1 P) Unmittelbar nötige Datenübernahme und Quittierung bleiben in der ISR, längere Auswertung und Textformatierung kommen ins Hauptprogramm. (1 P)
4. Nein. Es kommen $1/0{,}0008 = 1250$ Werte pro Sekunde an, aber selbst ohne Interrupttaufwand sind höchstens tausend auswertbar. Die wartende Menge wächst; ein endlicher Puffer verschiebt nur den Überlauf. Nötig sind etwa weniger Daten, kürzere Auswertung oder zulässiges Zusammenfassen. (1 P)

## Aufgabe 6: Ereignisse sicher übernehmen (5 Punkte)

Ein Hart, keine verschachtelten Interrupts. Ein Zähler hält die Anzahl noch zu bearbeitender Ereignisse fest und steht anfangs auf zwei. Einzelne Wortzugriffe sind im Plattformmodell unteilbar, die folgende Gesamtfolge nicht.

| Schritt | Hauptprogramm | ISR |
|---|---|---|
| 1 | liest Zähler zwei in einen lokalen Merker | |
| 2 | | erhöht den Zähler auf drei |
| 3 | beendet ein Ereignis und schreibt Merker minus eins zurück | |

1. Welcher Zählerstand entsteht, welcher wäre nach einem bearbeiteten Ereignis richtig? Behebt `volatile` allein den Fehler? Begründen Sie. (1 P)
2. Skizzieren Sie eine kurze geschützte Folge, die den gesamten Zählerstand lokal übernimmt und den gemeinsamen Zähler auf null setzt. Wo liegt die längere Verarbeitung? (2 P)
3. Warum wird danach der vorherige Freigabestand wiederhergestellt, statt grundsätzlich alle Interrupts einzuschalten? (1 P)
4. Die Anforderung ändert sich: Alle einzelnen Messwerte müssen erhalten bleiben. Reicht ein Ereigniszähler? Begründen Sie die benötigte Datenablage. (1 P)

**Musterlösung 6**

1. Es entsteht eins statt zwei: Ein Ereignis geht in der Zählung verloren. `volatile` macht die mehrschrittige Änderung nicht atomar. (1 P)
2. Vorherigen Freigabestand sichern, betreffende Interrupts sperren, Zähler lokal übernehmen und auf null setzen, Freigabestand wiederherstellen. Die lokal übernommene Anzahl wird danach außerhalb der Sperre verarbeitet. (2 P)
3. Eine bereits zuvor gewollte Sperre darf nicht unerwartet aufgehoben werden. (1 P)
4. Nein. Ein Zähler erhält nur die Anzahl, nicht die Werte. Benötigt wird ein ausreichend großer Datenpuffer mit konsistenter Übergabe und einer Regel für Überlast beziehungsweise Überlauf. (1 P)

## Optional: Terminrechnung als C-Funktion prüfen

Implementieren Sie die Zukunftsterminrechnung aus Aufgabe 4 als reine C-Funktion. Prüfen Sie bei Starttermin 240000 und Periode 30000 die now-Werte 239999, 240000 und 346000. Erwartet: 240000, 270000 und 360000. Ein Simulator ist dafür nicht nötig. Die Hardwarezugriffe aus den Vertiefungsfolien sind keine Voraussetzung für diese Prüfung.